

modified by the transaction. In this scheme, the height of a transaction is represented by a tuple, *txNumber*, which is the height of the transaction within the block.

The following is an illustration of a sample read-write set prepared by simulating a hypothetical transaction. In the following example, the use of incremental numbers represents the versions.

```
<TxReadWriteSet>
  <NsReadWriteSet name="chaincode1">
    <read-set>
      <read key="K1", version="1">
      <read key="K2", version="1">
    </read-set>
    <write-set>
      <write key="K1", value="V1">
      <write key="K3", value="V2">
      <write key="K4", isDelete="true">
    </write-set>
  </NsReadWriteSet>
</TxReadWriteSet>
```

Additionally, if the transaction performs a range query during simulation, the range query as well as its results will be added to the read-write set as query information.

### Transaction validation using a read-write set

A committer uses the read set portion of the read-write set for checking the validity of a transaction and the write set portion of the read-write set to update the versions and the values of the affected keys.

In the validation phase, a transaction is considered valid if the version of each key present in the read set of the transaction matches the version for the same key in the world state – assuming that all the preceding valid transactions, including the preceding transactions in the same block, are committed. An additional validation is performed if the read-write set also contains one or more query information.

This additional validation should ensure that no key has been inserted, deleted or updated in the super range, i.e., the union of the ranges of results captured in the query information. In other words, if we re-execute any of the range queries that the transaction performed during simulation while validating the committed state, it should yield the same results that were observed by the transaction at the time of simulation. This check ensures that if a transaction observes phantom items during commit, the transaction should be marked as invalid. This phantom protection is limited to range queries, i.e., the *GetStateByRange* function in the chaincode that is not yet implemented for other queries – the *GetQueryResult* function in the chaincode. Other queries are at risk of phantoms and

should therefore only be used in read-only transactions that are not submitted to ordering, unless the application can guarantee the stability of the result set between simulation and validation/commit time.

If a transaction passes the validity check, the committer uses the write set for updating the world state. In the update phase, for each key present in the write set, the value in the world state for the same key is set to the value as specified in the write set. Further, the version of the key in the world state is changed to reflect the latest version.

The following example helps with understanding the semantics. For the purpose of this example, the presence of a key, *k*, in the world state is represented by a tuple (*k*, *ver*, *val*) where *ver* is the latest version of the key *k* having *val* as its value.

Now, consider a set of five transactions T1, T2, T3, T4 and T5, all simulated on the same snapshot of the world state. The following snippet shows the snapshot of the world state against which the transactions are simulated, and the sequence of read and write activities performed by each of these transactions. Also note that transactions with multiple read-write sets are not yet supported.

```
World state: (k1,1,v1), (k2,1,v2), (k3,1,v3), (k4,1,v4),
(k5,1,v5)
T1 -> Write(k1, v1'), Write(k2, v2')
T2 -> Read(k1), Write(k3, v3')
T3 -> Write(k2, v2'')
T4 -> Write(k2, v2'''), read(k2)
T5 -> Write(k6, v6'), read(k5)
```

Now assume that these transactions are ordered in the sequence of T1,...,T5, which could be contained in a single block or different blocks:

- T1 passes validation because it does not perform any read. Further, the tuples of keys *k1* and *k2* in the world state are updated to (*k1,2,v1'*), (*k2,2,v2'*).
- T2 fails validation because it reads a key, *k1*, which was modified by a preceding transaction T1.
- T3 passes the validation because it does not perform a read. Further, the tuple of the key, *k2*, in the world state is updated to (*k2,3,v2'*).
- T4 fails the validation because it reads a key, *k2*, which was modified by a preceding transaction T1.
- T5 passes validation because it reads a key, *k5*, which was not modified by any of the preceding transactions. **END** 🐧

By: Swapnil Kulkarni

The author is an open source enthusiast with experience in blockchain, cloud native solutions, containers and enterprise software product architectures. He has diverse experience with private as well as hybrid cloud architectures. He is a technology hobbyist and writes at [cloudnativetech.wordpress.com](https://cloudnativetech.wordpress.com).